

PROJECT SOUNDING LINE

The Software Verification and Parallel Test Infrastructure System

PART III

Repository Policy, Codex Governance, and Release Assurance

Version 1.2 Amendment Edition

Amendment purpose

Turn Sounding Line v1.2 efficiency and conformance requirements into permanent repository law. Version 1.2 establishes the canonical effective-authority index, future-project inheritance, runtime-conformance enforcement inside the existing protected decision, gate-class policy, merge-lane boundaries, record-only closure, semantic carry-forward obligations, performance incidents, and Codex constraints so no future project can drift back to redundant full gates or globally serialized validation as normal practice.

Effective August 11, 2026 upon protected-mainline acceptance | Document ID CS-SL-P3-001 v1.2

A. Document Control and Amendment Authority

Field	Value
Program	Project Sounding Line
Subsystem	The Software Verification and Parallel Test Infrastructure System
Document	Governing Architecture and Product Requirements Document - Part III, Version 1.2 Amendment Edition
Base authority	Version 1.0, July 28, 2026, plus the accepted Version 1.1 Amendment Edition dated August 10, 2026.
Version	1.2
Effective date	August 11, 2026 upon acceptance through the protected Sounding Line mainline path.
Document ID	CS-SL-P3-001 v1.2
Status	Governing amendment candidate; becomes authoritative on protected-mainline acceptance. Additive and superseding only where clauses are explicitly named.
Amendment trigger	Observed growth in authoritative-run wall time and repeated setup/revalidation during the August 2026 concurrent project wave, including suite-per-runner setup repetition, broad exclusive serialization, browser provisioning duplication, and late expensive failure discovery.
Primary objective	Maximize verification throughput without reducing coverage, evidence integrity, source binding, risk floors, or release confidence.

Normative incorporation rule

Version 1.0 and Version 1.1 remain in force except where this Version 1.2 amendment explicitly adds, clarifies, or supersedes a clause. Version 1.2 is not permission to run fewer required tests or accept weaker evidence. It requires Sounding Line to obtain the same or stronger decision confidence with the least redundant execution that can be proven correct. Where Version 1.1 identifies overvalidation and manual coordination as incidents, Version 1.2 makes runtime conformance, gate scope, inheritance, and preservation enforcement explicit repository requirements rather than best-effort behavior.

The three-document authority split remains unchanged: Part I defines verification meaning, Part II defines execution machinery, and Part III defines repository, Codex, CI, mainline, and release enforcement.

1. Amendment Summary and Supersession Map

Existing area	Version 1.2 addition
2 Developer obligations	Adds obligation to preserve valid evidence, use the effective Sounding Line authority index, and avoid project-owned global test infrastructure.
3 Codex obligations	Adds mandatory v1.2 inherited verification contract, no redundant full gates, no resource overprovisioning, and no mainline-hold-as-general-work-hold behavior.
4 Change-to-test rules	Requires semantic invalidation manifest and gate-scope ceiling before broad rerun.

Existing area	Version 1.2 addition
10 Release-gate classes	Adds formal record-closure gate and runtime-conformance obligation inside mainline/release gates.
11 Mainline policy	Serializes canonical merge ownership only; development and non-conflicting validation remain parallel.
12 Release decision	Retains one protected authority: Sounding Line / Mainline Decision; conformance becomes mandatory internal evidence, not a second gate.
19-20 Performance / slow suites	Adds throughput SLOs, setup-to-execution reporting, runner-count/queue exposure, and conformance incident triggers.
21 Historical result governance	Makes carry-forward automation required steady-state functionality; conservative fallback is explicit debt.
26 Codex prompt contract	Every future phase inherits current effective Sounding Line authority automatically.
27 Completion report	Adds runtime/conformance/critical-path/preservation/overvalidation fields.
28 Incident policy	Adds RUNTIME_CONFORMANCE_INCIDENT and OVERVALIDATION_POLICY incidents.

2. Canonical Effective Sounding Line Authority Index

The repository SHALL contain one machine-readable authority index that tells every developer, Codex task, CI job, and future project which Sounding Line document versions and enforcement features are currently effective. This prevents prompts from depending on remembered PDF filenames or stale chat context.

```
testing/sounding-line-authority.json
{
  "authority": "SOUNDING_LINE",
  "partI": "1.2",
  "partII": "1.2",
  "partIII": "1.2",
  "requiredCheck": "Sounding Line / Mainline Decision",
  "runtimeConformanceRequired": true,
  "semanticCarryForwardRequired": true,
  "recordClosureGateRequired": true
}
```

2.1 Authority-index rules

- The index becomes authoritative only through protected mainline acceptance.
- CI policy validation rejects references to an unavailable or mismatched document/policy version.
- A human-readable companion record in the governing-document directory SHOULD list the accepted Part I/II/III baseline and amendments for discoverability.
- Historical documents remain audit evidence; the index identifies the effective interpretation chain.
- No prompt, README, completion message, or temporary script may silently supersede the effective index.

3. Permanent Future-Project Inheritance

Every future Voyagewright project and phase that may modify the repository SHALL inherit the current effective Sounding Line authority. Project documents own product semantics; they do not create competing global test authority, scheduler, release gate, or evidence model.

3.1 Required project/phase verification declaration

- Protected contracts and product risks introduced/changed by the phase.
- New/changed tests and their Sounding Line suite ownership.
- Fixture/baseline needs and declared resources.
- Browser/provider/environment requirements.
- Expected migration/privacy/security/accessibility obligations where applicable.
- Any project-specific manual/owner acceptance gate that supplements but does not replace Sounding Line evidence.

3.2 Project-owned infrastructure prohibition

A feature project MAY add an adapter or suite registration needed to prove its own behavior. It MUST NOT independently redesign global scheduling, branch protection, evidence carry-forward, generic database provisioning, browser-lane ownership, or finalizer semantics unless the task is explicitly operating under Sounding Line infrastructure authority to repair an INFRASTRUCTURE_DEFECT.

4. One Protected Authority, Conformance Inside It

The repository SHALL retain one stable protected mainline authority: `Sounding Line / Mainline Decision`. Runtime conformance is a mandatory obligation consumed by that decision; it is not a second competing required check with separate release semantics.

```
Sounding Line / Mainline Decision
requires:
  mandatory product evidence
  + security/privacy/accessibility/migration evidence as selected
  + fresh/preserved/rebound evidence completeness
  + runtime.conformance receipt
  + cleanup/resource-release proof
  + exact source/policy/plan identity

-> RELEASE_GO or RELEASE_NO
```

4.1 What conformance may block

- Evidence-integrity, source-binding, resource-isolation, case-count, or prepared-artifact identity defects are hard blockers.
- A planner that omits the required invalidation manifest or illegally uses stale carry-forward is a hard blocker.
- A record-only gate that selects broad executable proof without justification is a policy failure requiring plan correction.
- Pure external queue slowness is not a product or conformance failure; it is telemetry and may trigger an infrastructure incident.

5. Formal Gate Policy

Gate	Use	Repository rule
local-change	Default developer/Codex feedback after ordinary edits.	Impact-selected; nonauthoritative for merge.
subsystem	Project/subsystem closure evidence before protected mainline candidacy.	Complete subsystem proof according to current contracts.
mainline	Executable protected merge authority.	Minimum sufficient current repository evidence + runtime conformance.
record-closure	Post-merge record/status/provenance updates only.	Docs/catalog/receipt/index/policy integrity actually affected; no browser tax absent semantic

Gate	Use	Repository rule
		dependency.
release-candidate	Deep pre-release acceptance.	Full current release obligations; provider/owner/manual gates as required.

5.1 Gate escalation

A narrower gate may escalate only when semantic impact requires broader proof. A broader gate may not be selected merely because it is already scripted. Every escalation must be visible in the plan with a dependency/risk explanation.

6. Codex v1.2 Verification Contract

Every future Codex project/phase prompt that can reach mainline SHALL include or inherit the following contract from the effective Sounding Line authority index.

SOUNDING LINE V1.2 CONTRACT

- Use the current effective Sounding Line authority from testing/sounding-line-authority.json.
- Register and maintain tests with the protected contracts they prove.
- Let Sounding Line plan impact, resources, scheduling, evidence, and final decisions.
- Do not launch duplicate full gates.
- Do not treat SHA movement as blanket evidence invalidation.
- Do not prepare undeclared expensive resources merely because a generic script can.
- Do not widen timeouts/retries before failure classification.
- Enter CLOSURE_FROZEN when feature scope is materially complete.
- After exact-candidate RELEASE_GO and no material drift, merge promptly.
- Post-merge proof targets the actual integrated SHA and only the obligations required by policy.
- Record-only closure uses record-closure.
- A canonical merge hold is not a general development or validation hold.
- Do not start the next same-project phase before current phase is accepted in mainline.

Prompt inheritance beats prompt duplication

Future phase prompts do not need to reproduce the entire Sounding Line constitution. They must point to and obey the effective authority index. If a prompt conflicts with accepted Sounding Line policy, Sounding Line controls.

7. Canonical Merge Lane: Serialize Acceptance, Not Development

The continuous-development standard and Sounding Line merge policy SHALL distinguish canonical mainline ownership from ordinary project work. Exactly one candidate may own the final merge/integrated-SHA transition at a time, but that ownership does not freeze unrelated development or independently available validation resources.

Activity	Policy
Canonical merge action	Serialized. One holder reconciles exact main, obtains required decision, merges, proves integrated SHA, releases hold.
Project implementation	Parallel. Other phases continue safe current-phase work in their branches/worktrees.
Focused/unit/component validation	Parallel when resources/dependencies permit.

Activity	Policy
Browser/build/database work	Delayed only by the actual Sounding Line resource lease/capacity conflict, not by the mere existence of a merge hold.
Next same-project phase	Prohibited until current phase is accepted on mainline.

Merge-hold scope violation

A message such as “Project A owns the canonical merge hold, therefore all other projects must stop unrelated local/browser/build work” is nonconformant unless the blocked work requires an actually occupied Sounding Line resource.

8. Full-Gate Authorization and Duplicate-Run Prohibition

Situation	Required action
Initial executable mainline candidate	One authoritative mainline decision is normal after focused/subsystem proof and current-main reconciliation.
Product code changed after authoritative result	Replan semantic impact; rerun invalidated obligations. New full gate only if current mainline obligations require it.
Test-only/fixture repair	Rerun owning/consumer evidence; full unrelated matrix forbidden without manifest explanation.
Record-only change	Use record-closure. Full executable gate normally prohibited.
Unrelated mainline advance	Reconcile + invalidation manifest + preserve/rebind unaffected evidence; rerun only affected/integration obligations.
Third full gate with no production-code change	Mandatory OVERVALIDATION_POLICY incident review before another full gate.

8.1 Duplicate-run detector

CI/policy tooling SHOULD detect authoritative full-gate requests for the same phase where production-source fingerprint and required obligations are unchanged. A duplicate request must carry a new material invalidation reason. Optional confidence reruns after RELEASE_GO are prohibited.

9. Record-Only Closure Enforcement

The repository SHALL implement and use a dedicated record-closure gate for non-executable completion work. This converts the Version 1.1 policy “no browser tax for paperwork” into a concrete gate rather than a conversational convention.

Record-only change	Normal proof
Feature Catalog state promotion	Feature sync/validate, catalog tests, generated index checks, docs/policy validation as affected.
Completion receipt adds merge SHA/digests	Receipt schema/integrity/reader validation + docs.
Project status/changelog/index	Docs/generated-index checks; no product matrix unless executable semantics changed.
Evidence anchor projection	Evidence schema/digest integrity.
Sounding Line authority-index update	Policy/doc/version integrity + Sounding Line conformance/self-tests; broader runtime proof if enforcement behavior changes.

10. Mainline Advance and Evidence Carry-Forward Enforcement

When main advances while a phase is queued or closing, Codex SHALL reconcile current main and invoke semantic invalidation. The inability of current tooling to preserve evidence automatically is Sounding Line implementation debt, not permission to label every receipt invalid.

```
candidate H proven against main M0
main advances to M1 by unrelated project
H reconciles M0..M1

manifest:
  unit.helm          PRESERVED
  component.helm     PRESERVED
  browser.helm       PRESERVED
  unit.feature-catalog INVALIDATED
  docs/status        REBOUND
  exact integration   INVALIDATED

run INVALIDATED/REBOUND only -> finalizer
```

10.1 Fail-closed fallback

If the preservation engine cannot safely determine dependency equivalence, it may choose CONSERVATIVE_FALLBACK and rerun broader evidence. That decision must include the unsupported dependency relation, expected implementation fix, and incident/debt record. The project must not silently normalize the broad rerun as the new standard.

11. Repository Ownership of Sounding Line Infrastructure Changes

Changes to generic workers, planner, finalizer, resource broker, carry-forward engine, gate definitions, authority index, or runtime-conformance logic are Sounding Line infrastructure work. A feature project encountering an infrastructure defect may diagnose and report it but may not casually rewrite global verification behavior inside its closure branch.

- Infrastructure repair should use a dedicated Sounding Line branch/task unless the fix is an inseparable tiny correction explicitly owned by Sounding Line and proven independently.
- Runtime changes require Sounding Line self-tests, policy tests, conformance scenarios, and impacted adapter-family proof.
- A feature project may continue safe work while infrastructure repair proceeds; it must not bypass the required check.
- Accepted runtime changes become effective for all future projects through the authority index and machine-readable policy, not by copying text into each project.

12. Throughput SLOs and Incident Policy

Throughput targets are engineering service objectives. They expose control-plane defects but do not reduce proof requirements.

Metric/SLO	Target	Response
Warm mainline authoritative wall time	10-15 minute median target once v1.2 runtime is fully deployed.	Warning/diagnostic if exceeded; investigate queue/setup/critical path.
Cold mainline authoritative wall time	15-25 minute target under normal hosted availability.	Warning/diagnostic; cache miss and runner scarcity

Metric/SLO	Target	Response
		attributed separately.
Narrow TEST_DEFECT repair proof	3-8 minute target when only one/few suites invalidate.	If system reruns broad unrelated proof, open overvalidation review.
Unrelated-main reconciliation proof	3-8 minute targeted target when most evidence preserves.	Low preservation ratio without dependency change triggers invalidation review.
Record-only closure	2-5 minute target.	Browser/build selection without semantic explanation is a gate-scope violation.
Closure latency	>60 min from CLOSURE_FROZEN to merge-ready with no product defect.	Closure-latency warning; attribute queue/setup/execution/rework/drift.
Repeated authoritative gates	Third full gate with no production-code change.	Mandatory OVERVALIDATION_POLICY incident before another full gate.
Setup dominance	Setup time repeatedly exceeds suite execution for a worker class.	Runtime-conformance/performance review; consider bundling/prepared artifact.

12.1 Incident types

Incident	Meaning
RUNTIME_CONFORMANCE_INCIDENT	Generic runtime violates resource/scheduling/evidence-conformance rules.
OVERVALIDATION_POLICY	Broad/duplicate rerun occurs without semantic invalidation reason.
CLOSURE_REVALIDATION_INCIDENT	Verification churn dominates closure without increased confidence.
RESOURCE_BROKER_INCIDENT	Manual/shared-lane diplomacy or starvation replaces broker behavior.
REFERENCE_ENVIRONMENT_INCIDENT	Hosted/reference environment causes systematic timing/setup instability.

13. Completion Report Requirements v1.2

Field	Required value
Effective Sounding Line versions	Part I/II/III and authority-index digest.
CLOSURE_FROZEN timestamp	When feature development stopped.
Authoritative gate executions	Count, source SHA, gate, reason for each.
Invalidation manifest	Digest; counts PRESERVED/REBOUND/INVALIDATED/CONSERVATIVE_FALLBACK.
Preservation ratio	How much eligible evidence was reused safely.
Queue/setup/execution/teardown/finalizer timing	Critical-path decomposition.
Physical workers / logical suites	Shows batching efficiency without hiding evidence boundaries.
Prepared artifact cache state	Dependency/browser/baseline hits, misses, rebuild reasons.
Runtime conformance result	PASS/FAIL plus any incident codes.
Merge authorization timestamp	When protected decision made candidate merge-eligible.
Merge timestamp	Used to detect avoidable post-authorization delay.

Field	Required value
Integrated-SHA proof	Exact actual-main source and targeted proof disposition.
Record-closure result	Dedicated narrow closure gate outcome.

14. v1.2 Rollout and Enforcement Sequence

The amendment becomes permanent through both documents and repository enforcement. Adoption should be incremental so each accepted improvement accelerates the remaining project wave.

Step	Action	Result
1. Accept documents + authority index	Add Part I/II/III v1.2 amendments and effective authority index through protected mainline.	Future work inherits new law even while some runtime features remain implementation debt.
2. Runtime telemetry/resource-aware prep	Add timings and stop undeclared DB/browser setup.	Immediate speed gain; conformance begins detecting regressions.
3. Prepared artifacts/baseline	Dependency/browser cache + certified SQLite baseline.	Reduce repeated environment setup.
4. Bundled workers/typed capacities	Reduce hosted job count; dual browser lanes; dynamic ports; build/browser overlap.	Reduce queue exposure and serialization.
5. Semantic carry-forward + record gate	Implement invalidation manifest, derived receipts, targeted scheduling, record-closure.	Eliminate repeated full-gate loops after narrow/mainline changes.
6. Close implementation debt	Conformance tests prohibit legacy generic overprovisioning/global mutex paths.	v1.2 runtime fully accepted.

15. Future Project Acceptance Additions

- Every future project/phase resolves the current effective Sounding Line authority before planning tests or closure.
- Project prompts inherit Sounding Line rather than inventing an independent test authority.
- Mainline merge ownership serializes only canonical integration, not unrelated development or resource-independent validation.
- No future project may normalize suite-per-VM, fixed-port global locks, universal browser installs, or generic DB setup when resource declarations do not require them.
- Record-only closure uses the dedicated gate.
- Unrelated mainline movement invokes semantic carry-forward, not blanket invalidation.
- The stable required check remains Sounding Line / Mainline Decision.
- Runtime conformance is mandatory evidence inside that check.
- Throughput incidents trigger Sounding Line infrastructure remediation, never weaker assertions.
- The next same-project phase remains prohibited until current phase is accepted in mainline.

Final Part III v1.2 rule

Sounding Line must remain the strict verifier, not become the reason development crawls. Make the efficient path the enforced path: one authority, explicit gate scopes, semantic preservation, resource-aware execution, parallelism where isolation is real, and immediate convergence when proof is complete.

R. References and Amendment Closure

- Project Sounding Line Part III - Repository Policy, Codex Governance, and Release Assurance, Version 1.0, July 28, 2026.

- Project Sounding Line Part III - Version 1.1 Amendment Edition, August 10, 2026.
- Companion Project Sounding Line Parts I and II, Version 1.0 and Version 1.1 amendments.
- Voyagewright Continuous Development and Mainline Integration Standard, current accepted version at implementation time.
- Current repository branch protection, Sounding Line workflows/policy/runtime, project-status/Feature Catalog records, and protected mainline at implementation time.
- August 2026 concurrent Tideglass, Admiralty, Drydock, Helm, Wakebook, Shipwright, and Deepwater integration/validation incident history as operational evidence.

All unaffected Version 1.0 and Version 1.1 clauses remain governing. Version 1.2 supersedes any interpretation under which broad full-gate repetition, generic overprovisioned workers, fixed-port global serialization, or prompt-by-prompt memory is an acceptable permanent mechanism when machine-readable Sounding Line policy can enforce the correct behavior directly.